

Neural Marching Cubes

ZHIQIN CHEN, Simon Fraser University, Canada

HAO ZHANG, Simon Fraser University, Canada

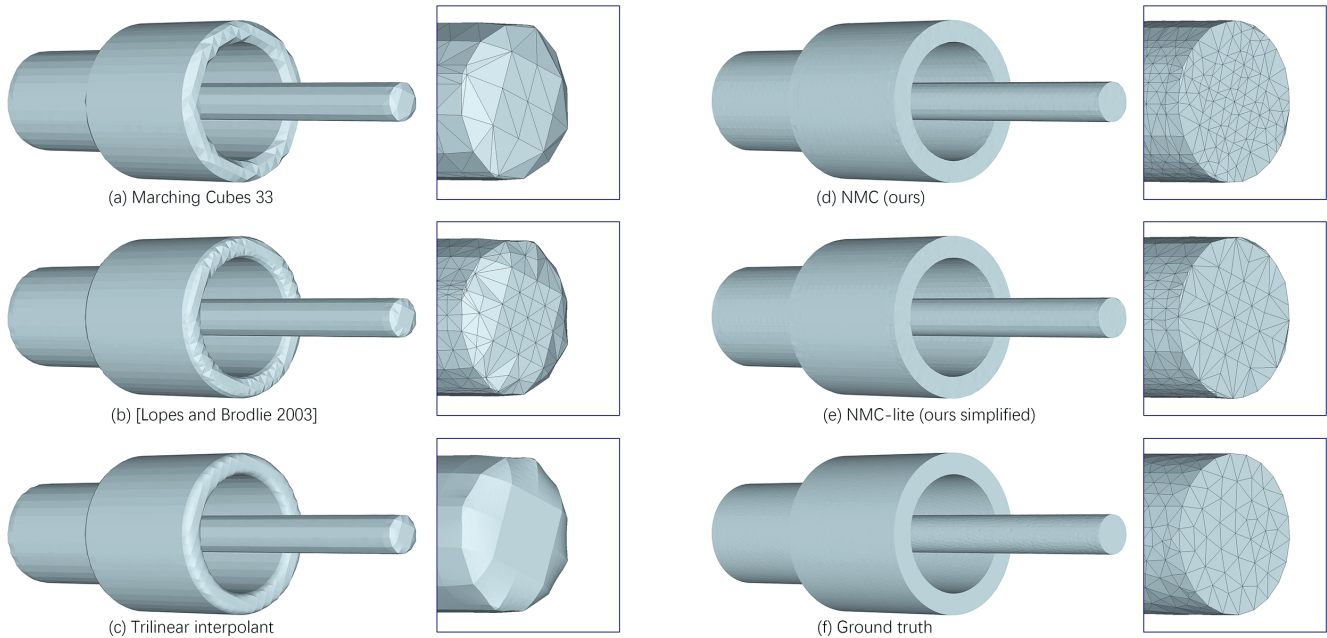


Fig. 1. Our Neural Marching Cubes (NMC) is trained to reconstruct the zero-isosurface of an implicit field, while preserving geometric features such as sharp edges and smooth curves. We compare NMC (d), and a simplified version (e), to the best-known MC variants (a-b), as well as a trilinear interpolant (c), none of which could reconstruct the features. The inputs to all methods are the same: a uniform grid of signed distances sampled from the ground truth (f).

We introduce *Neural Marching Cubes*, a *data-driven* approach for extracting a triangle mesh from a discretized implicit field. We base our meshing approach on Marching Cubes (MC), due to the simplicity of its input, namely a uniform grid of signed distances or occupancies, which frequently arise in surface reconstruction and from neural implicit models. However, classical MC is defined by coarse tessellation templates isolated to individual cubes. While more refined tessellations have been proposed by several MC variants, they all make heuristic assumptions, such as trilinearity, when determining the vertex positions and local mesh topologies in each cube. In principle, none of these approaches can reconstruct geometric features that reveal coherence or dependencies between nearby cubes (e.g., a *sharp edge*), as such information is unaccounted for, resulting in poor estimates of the true underlying implicit field. To tackle these challenges, we re-cast MC from a deep learning perspective, by designing tessellation templates more apt at preserving geometric features, and learning the vertex positions and mesh

topologies from training meshes, to account for *contextual* information from nearby cubes. We develop a compact per-cube parameterization to represent the output triangle mesh, while being compatible with neural processing, so that a simple 3D convolutional network can be employed for the training. We show that all topological cases in each cube that are applicable to our design can be easily derived using our representation, and the resulting tessellations can also be obtained naturally and efficiently by following a few design guidelines. In addition, our network learns local features with limited receptive fields, hence it generalizes well to new shapes and new datasets. We evaluate our neural MC approach by quantitative and qualitative comparisons to all well-known MC variants. In particular, we demonstrate the ability of our network to recover sharp features such as edges and corners, a long-standing issue of MC and its variants. Our network also reconstructs local mesh topologies more accurately than previous approaches. Code and data are available at <https://github.com/czq142857/NMC>.

Authors' addresses: Zhiqin Chen, Simon Fraser University, Burnaby, Canada, zhiqinc@sfu.ca; Hao Zhang, Simon Fraser University, Burnaby, Canada, haoz@sfu.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

0730-0301/2021/12-ART251 \$15.00

<https://doi.org/10.1145/3478513.3480518>

CCS Concepts: • **Computing methodologies** → **Shape modeling**.

Additional Key Words and Phrases: Surface reconstruction, isosurface, machine learning

ACM Reference Format:

Zhiqin Chen and Hao Zhang . 2021. Neural Marching Cubes. *ACM Trans. Graph.* 40, 6, Article 251 (December 2021), 15 pages. <https://doi.org/10.1145/3478513.3480518>

1 INTRODUCTION

The Marching Cubes (MC) algorithm [Lorensen and Cline 1987] is the most prominent method for isosurface extraction, and has been widely adopted in scientific visualization, shape reconstruction, and by the recent emerging approaches for learning neural implicit representations [Chen and Zhang 2019; Mescheder et al. 2019; Park et al. 2019]. MC takes as input a uniform grid of values representing a discretized implicit field, and extracts a triangle mesh representing the zero-isosurface of the field. The classical MC determines the local mesh topology and tessellation in each cube of the grid by examining the signs at the eight cube corners and referring to a predefined look-up table indexed by the sign configurations. If the isosurface intersects a cube edge, a new mesh vertex is added to that edge with its position computed via linear interpolation.

With its popularity and wide adoption, MC has seen notable improvements over the years. Marching Cubes 33 [Chernyaev 1995] is one of the first works to assume that the implicit field in each cube follows *trilinear interpolation* with respect to the cube vertices, and the ensuing meshing algorithm aims for topological correctness under the trilinearity assumption. This increases the number of unique cases of mesh tessellations from 15 (in the original MC [Lorensen and Cline 1987]) to 33, hence the name. The algorithm itself requires many tests to determine which topological case a cube belongs to, and the process can be error-prone. As a result, many patch-ups and improvements have been made to MC 33 [Custodio et al. 2013; Etienne et al. 2011; Lewiner et al. 2003; Lopes and Brodlie 2003]. Still, the trilinearity assumption, which has persisted, can lead to poor estimates of the true implicit field in general, and especially near *sharp features* of a shape, as shown in Figure 1.

Indeed, while MC has been employed for decades, its inability to recover sharp features, arguably its most long-standing issue, has not been fully resolved. Earlier tessellation templates [Chernyaev 1995; Lorensen and Cline 1987; Wyvill et al. 1986] were quite coarse and designed for reconstructing soft and smooth objects. Even with a refined tessellation to possibly represent sharp features in later follow-ups, e.g., [Lopes and Brodlie 2003], there is insufficient information in isolated cubes to disambiguate between soft patches and sharp edges, and this issue is worsened when the inputs are binary occupancies instead of signed distances. In general, a shape edge is not a “point-wise” feature, but a geometry property that reveals coherence or dependencies over neighboring cubes. Hence, edge prediction should account for that *context*, yet classical MC algorithms have not used such neighbor information.

In this paper, we introduce *Neural Marching Cubes* (NMC), a *data-driven* approach for isosurfacing from a discretized occupancy or signed distance field (SDF). The main premise of our work is that there is sufficient *predictability* in the vertex positions and local mesh topologies of “nice” mesh tessellations under the MC setting, in particular, when they reflect persistent features, such as sharp edges, over neighboring cubes. Hence, a well-designed learning approach would be more effective than handcrafting all the templates and making heuristic decisions such as trilinear interpolation. To this end, we re-design the tessellation templates so that they are more apt at preserving geometric features including sharp edges and corners, and develop a neural network to learn the vertex positions

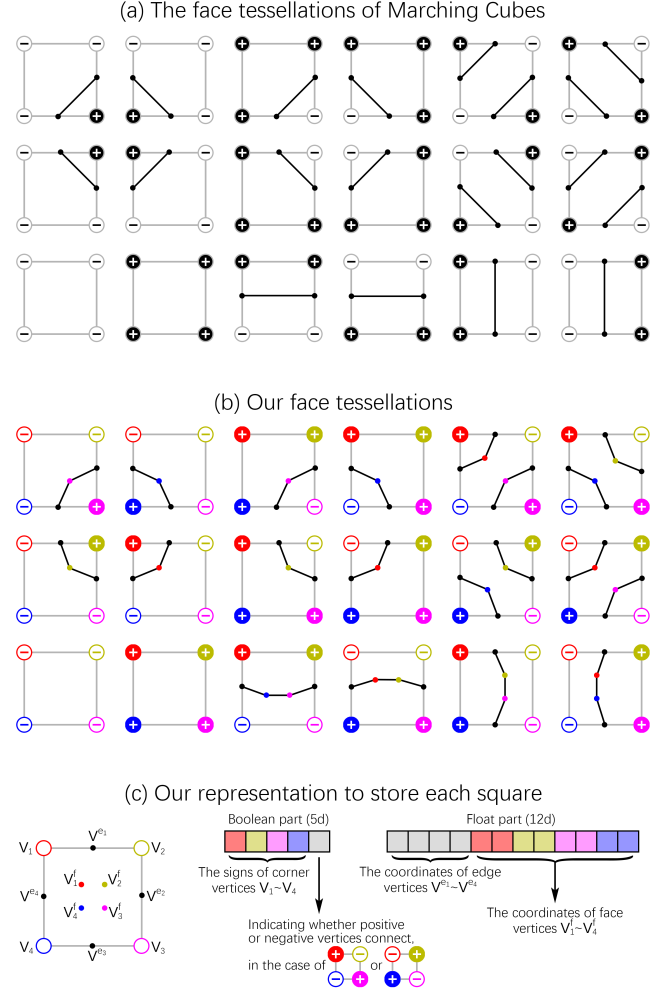


Fig. 2. Tessellation design (b) and parameterization (c) for NMC in 2D, in contrast to classical MC (a). Four new (face) vertices are added inside each square (c), each associated to a corner vertex (solid/hollow circle with +/- sign to indicate outside/inside), with matching color. Meshing information is encoded by a vector with a “boolean part” revealing topology and a “float part” storing all vertex positions; see Section 3.1 for more details.

and mesh topologies from a set of training meshes, so as to account for *contextual* information from nearby cubes.

To realize NMC, we must address several immediate challenges. First, we need a per-cube parameterization that is compatible with neural processing, so that the output mesh can be predicted by a network. Such a representation must contain all the information required to perform our modified MC algorithm, including mesh topology and vertex positions in each cell, while minimizing redundancy for efficient network training. Second, our reconfigured mesh tessellation templates must be *refined*, *complete* in the sense of avoiding ambiguities and covering all topological varieties, and be consistent with our defined representation. Last but not least, we must obtain quality training meshes resembling ideal outputs of NMC *automatically* from a collection of 3D shapes. These ground

truth meshes should use the designed tessellation templates in each cube, and be stored in the aforementioned representation.

Figure 2 illustrates our tessellation design and per-cube representation in the 2D case. In contrast to classical MC, we add new vertices to each cube, leading to more refined tessellations that can better represent geometric features. By carefully designing the representation (see Section 3), all topological cases that are applicable to our design, including all cases in MC 33, as well as all vertex positions, can be compactly encoded in a vector form for neural processing. Also, by associating the added vertices to their respective corner vertices, the new tessellations can all be obtained naturally and efficiently by following a few design guidelines.

With the above representation, our network for NMC is simply a 3D variant of ResNet [He et al. 2016] which inputs implicit field values. The network is trained with ground-truth meshes to set up both the topological and geometric losses, which operate respectively on the binary and float parts of the 3D version (see Figure 3) of the per-cube vector representation shown in Figure 2(c). By limiting its receptive field, our network learns local features, rather than from the entire shape, so that it generalizes well to new shapes and new datasets. Finally, we devise an optimization-based approach (see Section 3.4) to obtain the ground truth output meshes from raw 3D shapes based on our representation and tessellation design.

We evaluate NMC by qualitative and quantitative comparisons, in terms of reconstruction quality and feature preservation, to well-known MC variants, on both signed distance and binary voxel inputs. We show that our method is the first MC-based approach that is able to recover sharp features without requiring additional inputs other than a uniform grid of implicit field values. In addition, our network can more faithfully reconstruct local mesh topologies near thin shape structures and closeby surface sheets. We also provide a simplified version of NMC, which adopts the same mesh tessellation templates as [Lopes and Brodlie 2003], to study the fidelity-complexity trade-off. We finally show that our model can be trained to reconstruct clean meshes from noisy inputs by adjusting the training data, thus offering a useful tool for extracting 3D shapes from those shape representations designed for neural networks.

2 RELATED WORK

Our work is inspired by and closely related to Marching Cubes [Lorensen and Cline 1987] and its several variants. For completeness, we also discuss other algorithms for isosurfacing and emphasize the strengths of our NMC approach. Also relevant are recent works from the rapidly advancing field of neural geometry learning.

2.1 Marching Cubes (MC) and Variants

The original MC algorithm [Lorensen and Cline 1987] was proposed to reconstruct 3D structures from medical scan images, while a concurrent work [Wyvill et al. 1986] developed similar ideas for reconstructing soft objects. However, these methods did not guarantee surface consistency due to ambiguities of the tessellations in each cube; they may generate holes when tessellations in adjacent cells produce different corner connections on the common face [Dürst 1988]. Asymptotic Decider [Nielson and Hamann 1991] addressed the issue by assuming the implicit field within each face follows

bilinear interpolation with respect to the four face vertices, and proposed a solution to produce topologically correct tessellations under the bilinearity assumption. Several follow-up works [Chernyaev 1995; Matveyev 1994; Nielson 2003] further assumed the implicit field within each cube follows trilinear interpolation with respect to the eight cube vertices. Specifically, [Chernyaev 1995] was the first work to enumerate all possible topological cases with respect to the trilinear interpolant in the cube, and proposed Marching Cubes 33, which contains 33 unique cases under cube rotational and inversion symmetries (inverting all vertex signs), or 31 cases under rotation, mirroring, and inversion; see Figure 4(a). In comparison, the original MC algorithm had 15 and 14 cases, respectively.

While correctly enumerating all the topological cases does resolve ambiguities of the tessellations, the many tests required to identify specific topologies present a computational challenge. Lewiner et al. [2003] provided an efficient implementation of MC 33 by utilizing an extended look-up table, but left some unresolved issues [Etiene et al. 2011] which were tackled by follow-up work [Custodio et al. 2013]. Van Gelder and Wilhelms [1994] pointed out that to avoid non-manifold edges, the triangles in the tessellation templates should not lie in the face of a cube, as in MC 33, and this issue was addressed in later improvements [Cignoni et al. 2000; Lopes and Brodlie 2003]. Specifically, [Lopes and Brodlie 2003] introduced additional vertices on cube faces and interiors, but still followed the trilinearity assumption to place vertices, leading to poor estimates of surface features; see Figure 1. To extract sharp features from volume data, prior works typically required additional information, such as the positions and normals of the intersection points between cube edges and the shape, for vertex placement [Kobbelt et al. 2001]. A key point of NMC is that feature recovery is a *learnable* problem from training meshes. Once trained, our network can accurately predict sharp features and local mesh topologies without any additional input.

2.2 Other Isosurfacing Algorithms

Classic isosurfacing algorithms such as dual contouring [Ju et al. 2002] could preserve sharp features, but it requires the gradients of the intersection points on the edges of the grid cells, which could complicate the input setup. Dual contouring could also produce non-manifold edges, which is an issue addressed later by [Schaefer et al. 2007]. In dual marching cubes [Schaefer and Warren 2004], the implicit function is required as input and queried during reconstruction. There have also been other extensions to dual contouring [Dey and Levine 2008; Schreiner et al. 2006], including works [Varadhan et al. 2003; Zhang et al. 2004] which employ adaptive subdivision for simplification and efficient isosurfacing. Like dual contouring, all these methods require additional inputs such as the gradient information or the function of the underlying implicit field. By learning from training data, NMC can better preserve geometric features given only a uniform grid of sampled scalar values as input.

Marching Tetrahedra (MT) [Doi and Koide 1991] is another variant of MC: it splits each cube into tetrahedra to produce tessellations therein. The tessellation cases for MT are simpler than those of MC, but they were also not designed to recover sharp features. Finally, in a recent work called Analytic Marching [Lei and Jia 2020], rather than taking a signed distance field to mesh, the input is implemented

as a trained multi-layer perceptron (MLP) with rectified linear units (ReLU). The meshing is then performed by a marching over “analytic cells”, which correspond to linear regions resulting from a partitioning by the MLP. Overall, none of the above isosurfacing algorithms learn mesh tessellations from training data.

2.3 Neural Geometry Learning

With rapid advances in geometric deep learning, different neural representations have been proposed for 3D shapes. Voxels [Choy et al. 2016; Häne et al. 2017; Wu et al. 2016], point clouds [Achlioptas et al. 2018; Fan et al. 2017], and implicit models [Chen and Zhang 2019; Mescheder et al. 2019; Park et al. 2019] are among the most popular. But they all require a post processing step to extract a mesh. Deformable meshes/patches [Groueix et al. 2018; Wang et al. 2018, 2019] can directly output well-tessellated meshes, but they rely heavily on the input mesh templates and are unable to alter their topologies. There are only a handful of works [Chen et al. 2020; Dai and Nießner 2019; Deng et al. 2020; Gao et al. 2019] that could output polygonal meshes directly. In contrast, our method takes a discretized implicit field as input and directly outputs a triangle mesh, and therefore could act as the post processing step for many of the above representations.

NMC follows a recent trend in applying machine learning to classical low-level geometry processing tasks including mesh denoising [Wang et al. 2016], shape transform [Yin et al. 2019], point cloud upsampling [Yu et al. 2018], skeletonization [Lin et al. 2021], and subdivision [Liu et al. 2020], among many others. In particular, in neural subdivision, Liu et al. [2020] proposed a graph neural network to perform geometry-aware subdivision on triangle meshes. It recursively subdivides an input mesh by applying classic loop subdivision, while the positions of the newly added vertices are predicted by a neural network conditioned on local geometry.

In terms of combining machine learning and MC, two notable works, Deep Marching Cubes (DMC) [Liao et al. 2018] and MeshSDF [Remelli et al. 2020], both aim to make MC *differentiable*. Specifically, DMC learns a differentiable approximation of MC by predicting probabilistic occupancies and vertex displacements, while MeshSDF adopts a continuous model of how signed distance function perturbations locally impact surface geometry to obtain a differentiable surface parameterization. Another work, DefTet [Gao et al. 2020], shares a similar spirit as DMC, as it reconstructs tetrahedral meshes by predicting occupancies in an initial tetrahedral grid, and deforming the vertices to approximate the output shape.

Our work differs from DMC and MeshSDF in several major ways. First, our goals are different. The goal of DMC and MeshSDF is to directly obtain an explicit mesh representation from discrete raw inputs, e.g., point clouds, voxels, or images, in an end-to-end trainable manner, while NMC builds a framework to make MC learnable from training meshes. The input to NMC is a discrete implicit field of distances or occupancies obtained by any model, neurally or not. Second, the focus of DMC and MeshSDF is the end-to-end differentiability, while our focus is on designing and training a refined neural MC model to better reconstruct geometry and topology, in particular sharp features, unlike any other previous MC variant or differential extension. Case in point, DMC only adopted 8 out of

the 15 mesh tessellation templates from the original MC [Lorensen and Cline 1987] that have singly connected topologies, falling far short of [Lopes and Brodlie 2003] and NMC in terms of topological granularity. Last but not the least, DMC and MeshSDF rely on global features to predict the output shapes, not aiming to generalize to other shape categories not present in the training set. In contrast, our network employs a limited receptive field for each cube, leading to a more robust and general isosurfacing algorithm.

3 NEURAL MARCHING CUBES

We detail our representation for performing Neural Marching Cubes (NMC). We first introduce in a 2D example how the local topologies and tessellations in a square can be represented with a fixed-length code of binary values and float numbers; see Figure 2. Then we expand the representation into the 3D cube for NMC, as shown in Figure 3. We show how to design the mesh tessellations with respect to our representation and how our training data could be generated from raw meshes. Finally, we describe the network architecture and objective functions we designed to train NMC.

3.1 2D NMC: representation in a 2D square

We follow the common assumption in MC that if the two vertices of an edge in a cube (or a square) have different signs, there will be one and only one intersection point between the edge and the underlying zero-isosurface. As a result, all the situations in a square can be enumerated as in Figure 2(a). However, the tessellation templates in the classic MC algorithms are unable to represent geometric features such as sharp edges by design, hence they must be re-designed. Simply adding one vertex on each generated edge (black line) of the original templates could already solve the issue. But since we want a neural network to fully predict the meshing in each square, including the added vertices, we need to design a representation with a fixed format to store all the necessary information, so that the representation could be directly parsed into an output mesh, while being compatible with neural processing and training.

First, we add four face vertices ($v_1^f \sim v_4^f$) on the face, each associated with one corner vertex ($v_1 \sim v_4$), as shown in Figure 2(c) left. With the added vertices, new tessellations that can better preserve geometric features can be easily derived; see Figure 2(b).

Next, we design a fixed-length vector to fully encode the output mesh (edges) in each square; see Figure 2(c). The vector is split into a *boolean* or *binary part* to describe the topological cases, and a *float part* to store floating point numbers as vertex positions.

When the signs of the four corner vertices are given, there is only one ambiguity case, when both ends of a diagonal line are with the same signs but the ends of an edge are with different signs; see top-right corner of Figure 2(b). This ambiguity can be resolved by adding a face sign which is positive if the connected vertices are positive, and vice versa. Hence, the boolean part has 5 values storing the signs: one face sign and four signs for the corner vertices. On the other hand, we need to store all vertex positions in the float part, whether the vertices are being used or not. Since each edge vertex only has one degree of freedom, four edge vertices take 4 floats to store. Adding the 8 numbers for the 2D coordinates of the four face vertices, in total we have 12 numbers in the float part.

However, note that the representation for each square is not the same as the representation we use to predict the entire 2D shape, because of the redundancy: an edge vertex is shared by two adjacent squares, and a corner vertex is shared by four. Therefore, when representing the squares of an entire shape, we only need to store the sign of one corner vertex (v_1) and the face sign in the boolean part, and two edge vertices (v^{e_1}, v^{e_4}) and all four face vertices in the float part, leading to 2d and 10d vectors, respectively. Afterwards, a 2D Convolutional Neural network (CNN) could be applied to take as input an $M \times N$ array of implicit field values, and output an $M \times N \times 12$ array that is parsed into a 2D mesh.

3.2 3D NMC: representation in a 3D cube

As shown in Figure 3, we design the NMC representation for a 3D cube in a similar manner as its 2D counterpart shown in Figure 2. In addition to the edge vertices ($v^{e_1} \sim v^{e_{12}}$), we keep the four added face vertices for each of the six faces of the cube. We also add eight interior vertices ($v_1^c \sim v_8^c$) in the cube, each affiliated with one corner vertex ($v_1 \sim v_8$) of the cube. Details on how to properly tessellate the cube with these new vertices will be discussed in Section 3.3. In this section, we focus on how to represent the topological cases and the positions of the added vertices, using a boolean and a float part respectively, as shown in Figure 3(c).

For the boolean part, we require at least eight signs at the corner vertices of the cube and six face signs to describe or index the local mesh topology in each cube. However, these are not sufficient to resolve all ambiguities. As already observed in MC 33 [Chernyaev 1995], when there are two connected components with the same sign on the surface of a cube, the two could be connected with a tunnel inside the cube. The real situations could be far more complicated than that. There could be more than two connected components on the surface of the cube, and there could be more than one tunnel to connect the two components. To simplify the situation, we draw inspiration from the topological cases in MC 33, which are shown in Figure 4(a). Note that all the 33 cases have either zero or one tunnel, if there are exactly two connected components with the same sign. Therefore, we assume that in the case of two connected components, there could be one tunnel connecting the components, or none at all. In other cases with just one or more than two connected components, we assume zero tunnel. With such a simplifying assumption, we only need to add one binary value to indicate whether a tunnel exists, leading to a total of 15 binary values in the boolean part.

For the float part, we need to store 44 added vertices: 12 edge vertices, $6 \times 4 = 24$ face vertices, and 8 interior vertices, as shown in Figure 3(a-b). Since each edge vertex only has one degree of freedom, 12 edge vertices would only need 12 floats to store. Each face vertex has two degrees of freedom, therefore 24 face vertices take 48 floats to store. Plus the 24 floats for the 3D coordinates of the 8 interior vertices, in total we have 84 numbers in the float part.

However, similar to the 2D cases, the representation for each cube is not the same as the representation we use to predict the entire 3D shape, because of the redundancy: a corner vertex is shared by eight cubes, an edge vertex is shared by four, and a face vertex is shared by two. Therefore, when representing the cubes of an entire

shape, we only need to store 5 values in the boolean part (the sign of v_1, f_1, f_3, f_5 ; and the tunnel flag), and 51 values in the float part (3 edge vertices on edges e_1, e_4, e_9 ; 12 face vertices on faces f_1, f_3, f_5 ; and all 8 interior vertices). Afterwards, a 3D CNN can be applied to take as input an $M \times N \times P$ array of implicit field values, and output an $M \times N \times P \times 56$ array that could be parsed into a 3D mesh.

3.3 3D NMC: tessellating a 3D cube

In this section, we elaborate how we tessellate the cube with respect to each topological case. To facilitate the tessellation design, we developed a graphical user interface for interactive modelling, and employed the interface to design and render all the cases as shown in Figures 4 and 5.

Generally, the tessellation design needs to comply with several basic principles. First, the resulting mesh should contain only triangles. Second, the mesh should completely separate vertices with different signs, i.e., any path inside the cube that connects two vertices of different signs must intersect with the mesh. Third, there should be no non-manifold edges or vertices. Specific to our method, there is a fourth principle to follow: we are only allowed to use the vertices present in our cube representation as described in Section 3.2.

However, the very first step we need to take before designing the tessellations, is to enumerate how many unique topological cases there are. In our cube representation, we have 15 binary values to describe the cases, therefore we have a total of $2^{15} = 32,768$ cases. Yet, if we consider the presence of rotational symmetries, mirroring symmetries, inversion symmetries (inverting all vertex and face signs in a cube), and remove all invalid cases with respect to the tunnel flag, we have a total of 37 unique cases.

We can directly use the face tessellations in Figure 2 to tessellate the six faces of a cube, as shown in Figure 5(b). Afterwards, we follow several guidelines to create a mesh inside the cube: a) If there is a tunnel, then the tunnel must contain the small center cube made by the eight interior vertices; b) if there is no tunnel, then connect all available face vertices to their corresponding interior vertices; c) avoid long triangles. See Figure 6 for several examples.

The completed tessellations of our method can be found in Figure 5(a). The tessellation design allows much freedom and does not necessarily have to follow our guidelines. For example, we could simply take the tessellations in [Lopes and Brodlie 2003] into our framework, as show in Figure 4(b). Since this tessellation design employs fewer vertices and triangles, we coin our Neural Marching Cubes using this specific tessellation design as *NMC-lite*. Note that in both cases, the training data will be prepared and the network will be trained with respect to their own tessellation designs.

3.4 Data preparation

We now introduce data preparation for NMC, i.e., the preprocessing step to convert a raw mesh into a form compatible with our cube representation in Figure 3 for neural processing; it is an $M \times N \times P \times 5$ array for the boolean part and an $M \times N \times P \times 51$ array for the float part, when the input grid is $M \times N \times P$. We divide a raw 3D mesh into small cubes to process each separately, as in MC. For each cube, we first determine its topological case. Then we put the corresponding tessellation template inside that cube, and optimize the vertices of

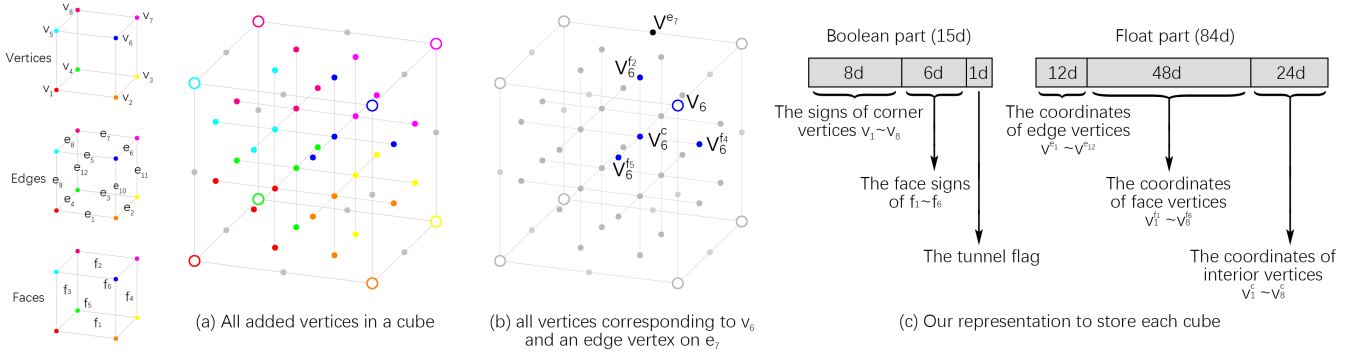


Fig. 3. Per-cube parameterization for our NMC in 3D, with 12 edge vertices (v_i^e), $6 \times 4 = 24$ new face vertices (v_j^f), along with 8 new interior vertices (v_k^i), as shown in (a). Vertices with the same color correspond to the same cube vertex, as shown in (b), where the grey lines in (a-b) are for ease of visualization only. The vector representation for local mesh topology (the boolean part) and vertex positions (the float part) is given in (c), where the number of floats needed to represent a vertex depends on the degrees of freedom, e.g., one for an edge vertex, two for a face vertex, and three for an interior vertex.

the tessellation template to minimize the Chamfer Distance with respect to the original mesh. An overview is given in Figure 7.

To determine the topological case in a cube, we compute a $9 \times 9 \times 9$ grid of signed distances inside the cube, so that each face contains 9×9 signed distances. We then check the connectivities between the vertices through the SDF grid, where adjacent grid points with the same sign are considered connected, to determine the case for each of the six faces. After the face cases are determined, we only need to test whether there is a tunnel to determine the cube case, which can be done by checking the number of connected components inside the cube. Note that in several situations, the cube cannot be represented with our templates, e.g., when there are two or more intersections on a cube edge, or when there are complex structures inside the cube that are unaccounted for.

We perform tests to validate whether an edge/face/cube can be represented using our templates by checking the number of connected components, which are compared against the templates in Figure 2 (for faces in the 2D case) or Figure 5 (for cubes in 3D). The edges/faces/cubes that do not have matching numbers are deemed to be invalid. For example, if the end vertices (grid points) of an edge are with different signs, then the 9 grid points on the edge should contain exactly two connected components, one positive and one negative. In a 3D cube, say Case 6.1.1, there are three connected components, one positive and two negatives, while in Case 6.1.2, there are two connected components because of the tunnel.

The removal of invalid edges/faces/cubes from the training meshes is critical to NMC and this is accomplished by a *masking* process. Specifically, we generate masks during data preparation to indicate valid values in our representation with **1**'s and invalid or irrelevant values with **0**'s, where invalidity implies that the edge/face/cube cannot be represented by our designed tessellation templates, and a value stored in our representation is irrelevant if it does not affect the output mesh (e.g., the face sign in an unambiguous face, or the tunnel flag in a cube that cannot form a tunnel). For shape s , we denote the input array as $I_s \in \mathbb{R}^{M \times N \times P}$, the array of the boolean part as $B_s \in \{0, 1\}^{M \times N \times P \times 5}$, and the array of the float part as $F_s \in [0, 1]^{M \times N \times P \times 51}$. Therefore, the mask of B_s is $M_{B_s} \in \{0, 1\}^{M \times N \times P \times 5}$ and the mask of F_s is $M_{F_s} \in \{0, 1\}^{M \times N \times P \times 51}$.

After the topological case is settled, we put the corresponding tessellation template inside the cube and optimize its vertices to approximate the original mesh. However, since adjacent cubes share edges and faces, we first determine the positions of all edge vertices, then all face vertices, and finally all interior vertices, to avoid repeated computations. Note that only the edge vertices do not require optimization since we can find them by checking the intersection points between the cube edges and the original mesh. Take the interior vertices for example – while the face vertices can be optimized in a similar way, we densely sample points on the mesh inside the cube to obtain a point cloud P . Denote the vertices, edges, and triangles in the tessellation template as V , E , and T , respectively. Denote the point-triangle Euclidean distance as $D(p, t)$, and the point-point Euclidean distance as $d(v_1, v_2)$, we have the objective function

$$L_{total} = L_{P \rightarrow T} + L_{T \rightarrow P} + \gamma L_{reg}, \text{ with} \quad (1)$$

$$L_{P \rightarrow T} = \frac{1}{|P|} \sum_{p \in P} \min_{t \in T} D(p, t), \quad (2)$$

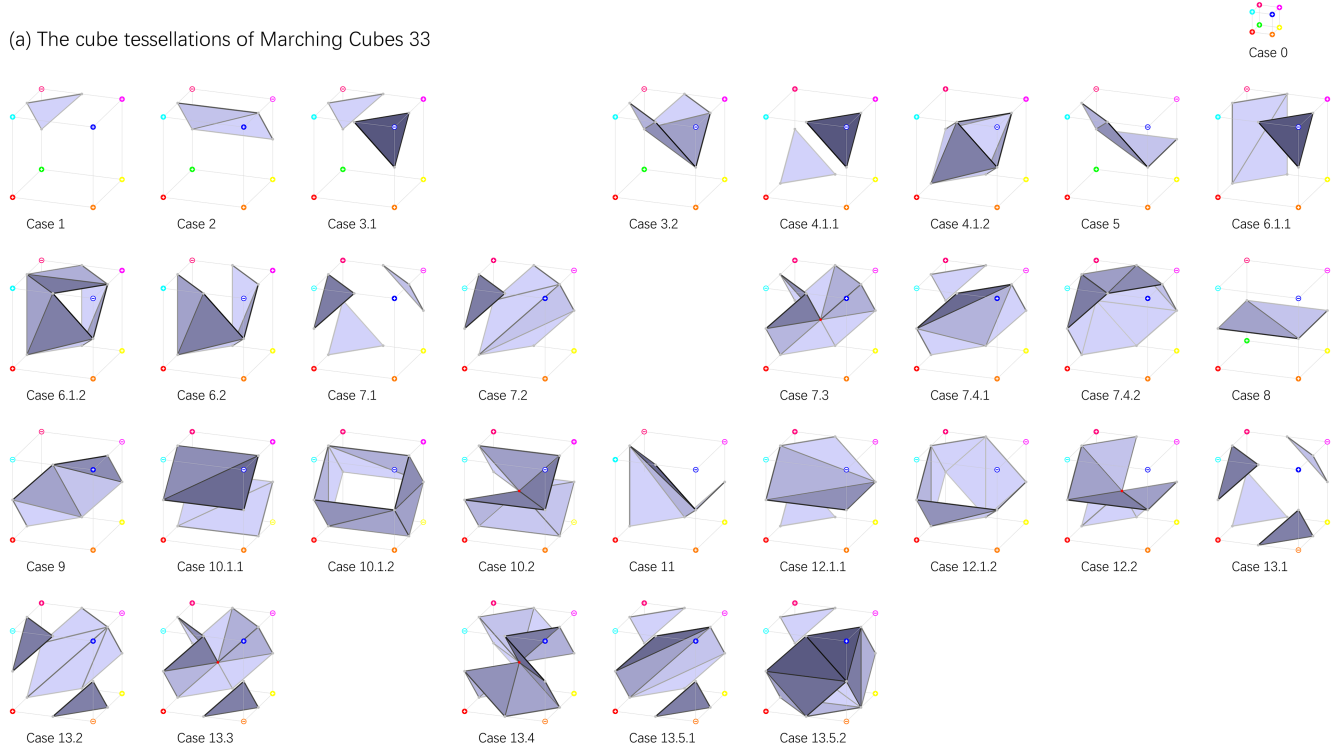
$$L_{T \rightarrow P} = \frac{1}{|T|} \sum_{t \in T} \min_{p \in P} D(p, t), \quad (3)$$

$$L_{reg} = \frac{1}{|V|} \sum_{v_1 \in V} \min_{\{v_1, v_2\} \in E} d(v_1, v_2), \quad (4)$$

where $L_{P \rightarrow T}$ is the point-to-triangle distance, $L_{T \rightarrow P}$ the triangle-to-point distance, and L_{reg} a regularization term to keep the surface as “tight” as possible by minimizing edge lengths, and γ is set to 0.1.

Note that it is possible to use the above objective function to train the network directly, instead of using a mean squared error loss as we will describe in the next section. However, to ensure the quality of the generated mesh, we usually sample a very dense point cloud to perform the optimization. The computational time and memory cost make it intractable to train the network directly with the optimization objectives.

(a) The cube tessellations of Marching Cubes 33



(b) The cube tessellations of [Lopes and Brodlie 2003], with our extended topological cases (indicated with a *)

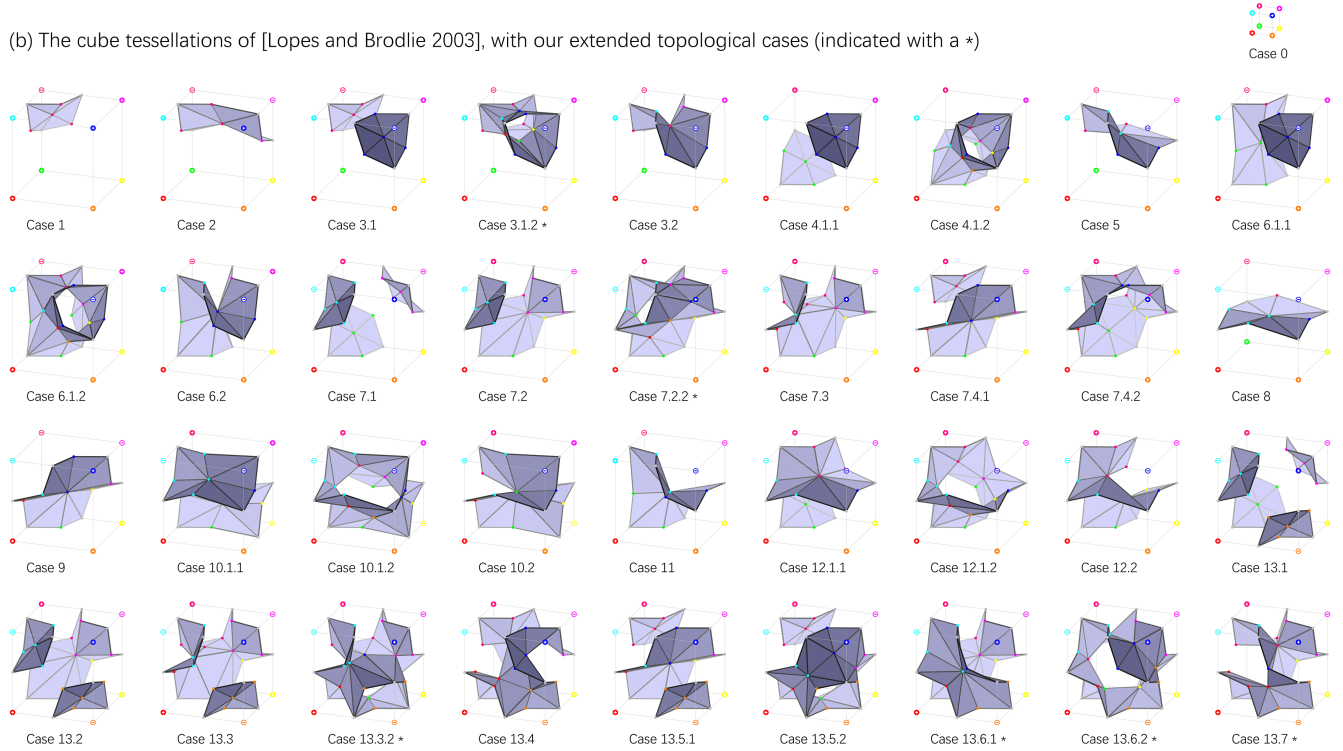
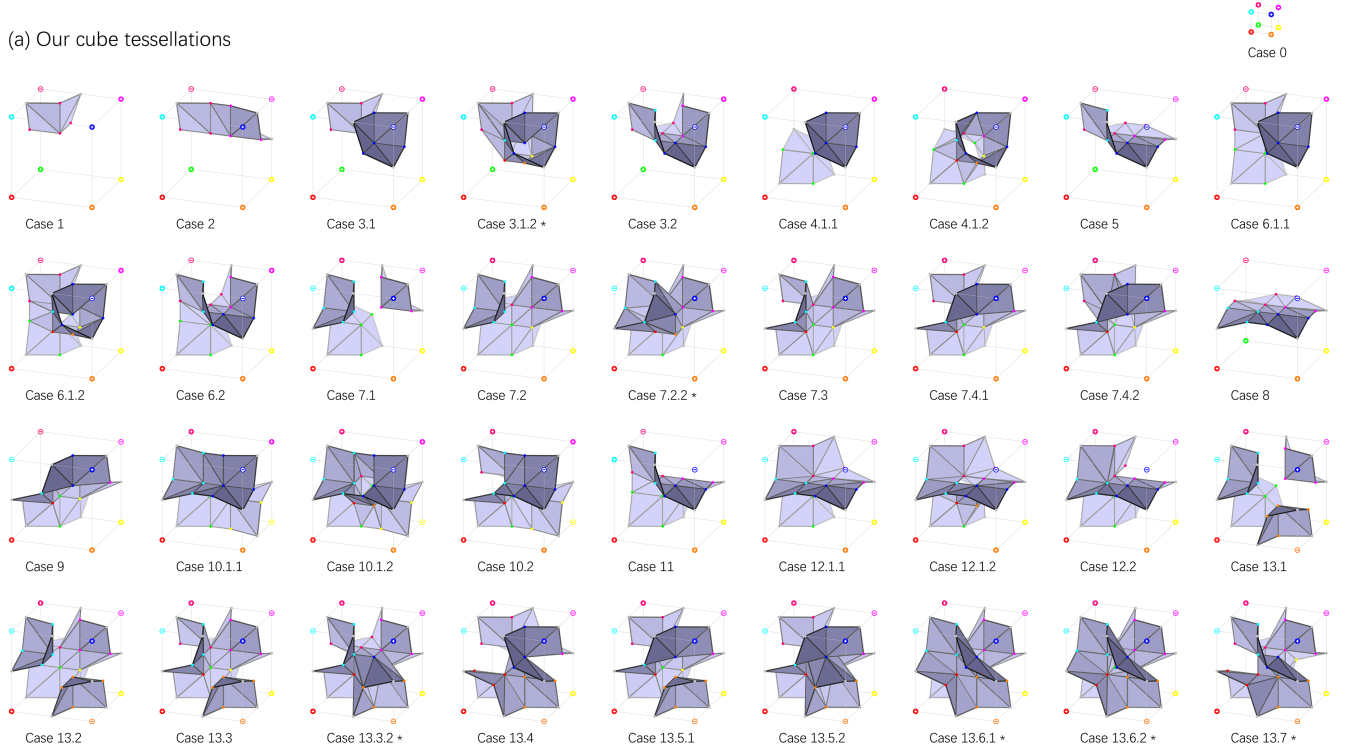


Fig. 4. The 3D cube tessellations of Marching Cubes 33 [Chernyaev 1995] and [Lopes and Brodlie 2003]. Note that they both present 31 cases, since Case 12.3 is equivalent to Case 12.2 and Case 14 is equivalent to Case 11, with respect to rotational and mirroring symmetries. In (b), we also add our extended topological cases to [Lopes and Brodlie 2003], indicated with a *, to form a simplified version of our NMC tessellations, denoted as NMC-lite.

(a) Our cube tessellations



(b) Our face tessellations



Fig. 5. Our cube tessellations and face tessellations for all the 37 unique topological cases considered by NMC, where vertices with the same color correspond. Note Case 0 in the top-right corner which indexes the case where all signs on the cube vertices are the same.

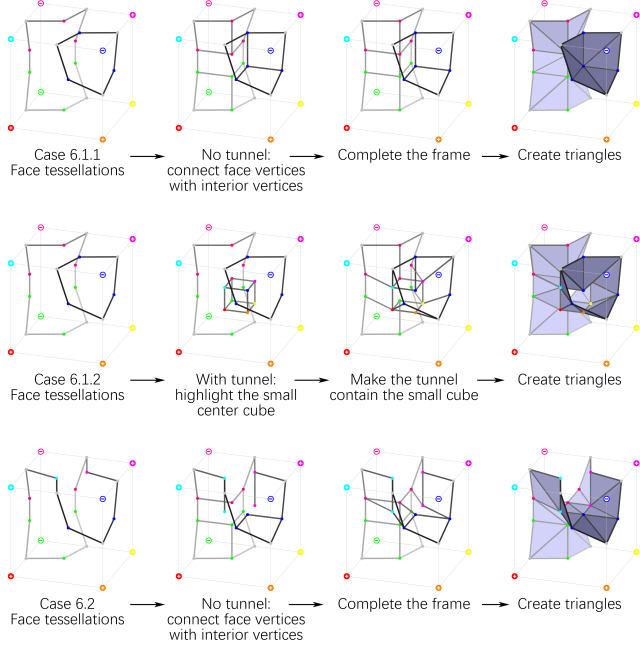


Fig. 6. Example tessellation steps for our NMC designs. The face tessellations in the first column follow Figure 2, therefore they are considered as “given”, and we only need to add new structures inside the cube.

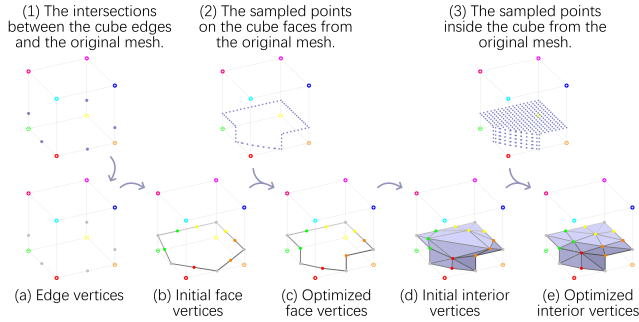


Fig. 7. Our preprocessing step to prepare the training mesh data for NMC. After determining the topological case for the cube, we optimize the vertex positions to approximate the original mesh. The initial face vertices are mid-points or trisection points, while the initial interior vertices in the cube are averages of connected edge vertices and face vertices.

3.5 NMC network and objective functions

The input to our network is an array of implicit field values I_s , and the ground truth outputs contain a boolean array B_s and a float array F_s . The masks M_{B_s} and M_{F_s} indicate which values in B_s and F_s are valid. Since they are all 3D arrays (with feature channels), we could directly apply 3D convolutional neural networks for the task. Specifically, we use a 3D variant of ResNet [He et al. 2016] as our backbone network, with receptive fields of size 7^3 .

For the objective functions, we use a binary cross entropy (BCE) loss for the boolean part and a mean squared error (MSE) loss for the float part. Denote the outputs of our network as $D_s = f_B(I_s)$ and $H_s = f_F(I_s)$ for the boolean part and float part, respectively,

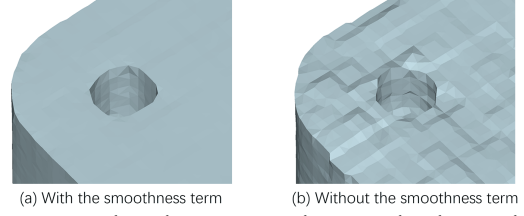


Fig. 8. Output meshes when our network is trained with vs. without the smoothness term when the inputs are binary voxel/occupancy grids.

and denote the entire shape dataset as $\mathcal{S}$. Let all multiplications in the following equations be element-wise products, then we have

$$L_{bool} = \mathbb{E}_{s \sim \mathcal{S}} \| -M_{B_s}(B_s \log(D_s) + (1 - B_s) \log(1 - D_s)) \|_1, \quad (5)$$

$$L_{float} = \mathbb{E}_{s \sim \mathcal{S}} \| M_{F_s}(F_s - H_s) \|_2^2. \quad (6)$$

We could directly sum L_{bool} and L_{float} with a weighting parameter to obtain the final objective function. However, our experiments showed that it is tedious to find an appropriate weight for the two terms. Therefore, we choose to use two distinct networks to predict D_s and H_s separately, so that one network is trained with L_{bool} and another with L_{float} without any interference.

However, the above settings are not sufficient for binary voxel inputs, due to considerable ambiguities of the possible shapes represented by the input voxels. Therefore, we use the aforementioned settings for SDF grid inputs, and make a few adjustments when the inputs are binary voxels. Specifically, we enlarge the receptive fields of our backbone network from 7^3 to 15^3 to reduce ambiguity, and add a smoothness term to the loss function on the float part,

$$L_{float}^* = L_{float} + \gamma L_{smooth}, \text{ with}$$

$$L_{smooth} = \mathbb{E}_{s \sim \mathcal{S}} \sum_{(u,v) \in E_s} \| \mathbb{1}(|F_s^u - F_s^v| < \sigma) \cdot (H_s^u - H_s^v) \|_2^2, \quad (7)$$

where E_s denotes the set of all edges (u, v) in the ground truth (GT) output mesh for shape s , $F_s^u \in \mathbb{R}^3$ is the coordinates of vertex u in the GT mesh, and $H_s^u \in \mathbb{R}^3$ is the coordinates of u in the *predicted* mesh. Note that the mesh tessellations of the GT mesh and the predicted mesh are the same since the tessellations are determined solely by the boolean part, and we use the GT boolean part when training the float part. In addition, $\mathbb{1}(\cdot)$ in the equation converts true/false into 1/0, respectively, and the parameters $\sigma = 0.0002$ and $\gamma = 10$ are fixed throughout our experiments.

Overall, the smoothness term encourages the output surfaces to align with the *coordinate axes*, with the underlying assumption that the GT surfaces generally share the same characteristic. We show the impact of L_{smooth} in Figure 8 and explain this choice in the Appendix by experimenting with different smoothness terms.

The network architectures and more training details can also be found in the supplementary material.

4 RESULTS AND EVALUATION

In this section, we show results and evaluate NMC both qualitatively and quantitatively, on both SDF and binary voxel inputs. We compare NMC to well-known MC variants, and demonstrate its generalizability and the ability to handle noisy input.

Table 1. Quantitative comparison results on ABC test set with SDF input.

128 ³ resolution	CD($\times 10^5$) $\downarrow$	F1 $\uparrow$	NC $\uparrow$	ECD($\times 10^5$) $\downarrow$	EF1 $\uparrow$	#V	#T
MC33	4.143	0.870	0.972	4.063	0.193	22,048.41	44,107.11
64 ³ resolution	CD($\times 10^5$) $\downarrow$	F1 $\uparrow$	NC $\uparrow$	ECD($\times 10^5$) $\downarrow$	EF1 $\uparrow$	#V	#T
MC33	4.850	0.788	0.950	5.736	0.105	5,472.51	10,953.67
Lopes2003	4.803	0.798	0.958	6.841	0.100	21,979.95	43,892.05
Trilinear	4.733	0.803	0.960	7.275	0.098	-	-
NMC-lite	4.341	0.877	0.975	0.382	0.759	22,710.56	43,876.87
NMC	4.323	0.877	0.975	0.390	0.758	42,766.54	85,543.83
32 ³ resolution	CD($\times 10^5$) $\downarrow$	F1 $\uparrow$	NC $\uparrow$	ECD($\times 10^5$) $\downarrow$	EF1 $\uparrow$	#V	#T
MC33	5.239	0.570	0.900	5.504	0.048	1,297.38	2,595.47
Lopes2003	5.343	0.577	0.911	6.213	0.047	5,215.12	10,397.68
Trilinear	5.161	0.585	0.915	7.217	0.045	-	-
NMC-lite	3.922	0.823	0.950	0.532	0.631	5,464.48	10,389.43
NMC	3.919	0.824	0.949	0.598	0.634	9,728.20	19,460.09

Table 2. Quantitative comparison on organic FAUST shapes with SDF input.

128 ³ resolution	CD($\times 10^5$) $\downarrow$	F1 $\uparrow$	NC $\uparrow$	ECD($\times 10^5$) $\downarrow$	EF1 $\uparrow$	#V	#T
MC33	4.533	0.985	0.984	0.892	0.383	12,551.21	25,076.50
Lopes2003	4.487	0.985	0.986	0.858	0.409	50,649.41	100,417.26
NMC-lite	3.696	0.992	0.987	0.559	0.628	50,205.72	100,401.08
NMC	3.706	0.992	0.987	0.625	0.628	83,023.47	166,036.10

Datasets. For our experiments, we mainly work with the first chunk of the ABC dataset [Koch et al. 2019], which consists of triangle meshes of CAD shapes. Such CAD shapes are characterized by their rich geometric features (e.g., sharp edges, corners, smooth curves, etc.) and topological varieties. We split the set into 80% training (4,280 shapes) and 20% testing (1,071 shapes). During data preparation, we obtain triangle meshes over 32³ and 64³ grids to train our network. Evaluation is conducted on the testing set. Later, to assess the generalizability of NMC, we also test (not train) the same network on the Thingi10K dataset [Zhou and Jacobson 2016], which contains a variety of 3D-printing models.

Evaluation metrics. To perform quantitative evaluation, we sample 100K points $S = \{s_i\}$ uniformly distributed over the surface of a shape, and then use Chamfer Distance (CD) and F-score (F1) to evaluate the overall quality of a reconstructed mesh, and Normal Consistency (NC) to evaluate the quality of its surface normals.

Inspired by BSP-Net [Chen et al. 2020], we employ Edge Chamfer Distance (ECD) and Edge F-score (EF1) to evaluate the preservation of sharp edges. We use the same “sharpness” definition in BSP-Net as $\sigma(s_i) = \min_{j \in \mathcal{N}_\epsilon(s_i)} |\mathbf{n}_i \cdot \mathbf{n}_j|$, where $\mathcal{N}_\epsilon(s)$ extracts the indices of the points in S within distance ϵ from s , and $\mathbf{n}_i$ is the surface normal at point s_i . We compute an “edge sampling” of the surface by retaining points for which $\sigma(s_i) < 0.2$. Given two shapes, the ECD and EF1 between them are simply the CD and F1 between the corresponding edge samples. We also count the number of generated vertices (#V) and triangles (#T) to reveal the fidelity-complexity trade-off.

Mesh extraction from SDF grids. We first compare the two versions of our method, NMC and NMC-lite, with the two best-known MC variants to date, Marching Cubes 33 [Lewiner et al. 2003] (MC33) and [Lopes and Brodlie 2003] (Lopes2003), on the task of mesh extraction from grids of SDF values. Quantitative comparison results are reported in Table 1, with two choices of input resolutions: 64³

and 32³, on the ABC test set. The results show that, with the same SDF inputs, our method outperforms MC33 and Lopes2003 on all the quantitative measures considered.

We also add a row (top row in Table 1) for MC33 with the input resolution upsampled to 128³. As we can see, even with 8 $\times$ the amount of input information as NMC and NMC-lite, MC33 *underperforms* on all measures except for CD. In terms of edge preservation, our method is superior. This is also verified by the visual results in Figure 11, comparing MC33 on 128³ input and NMC on 64³ input.

Figure 9 shows qualitative comparisons between the various methods, on sample inputs from the ABC test set and Thingi10K. We can observe that NMC, and to a lesser extent, NMC-lite, are the only methods that can recover sharp edges and corners, while the smooth curves are also well preserved. In fact, our method can reconstruct both sharp and soft edges well, as demonstrated in the last row. Furthermore, examples in the first row and the third row (at a smaller scale) exhibit thin structures in a shape, which causes both MC33 and Lopes2003 to produce incorrect local topologies, due to the trilinear interpolant assumption. On the other hand, our method infers the correct topological cases — the ambiguous Case 10.1.1 (see Figure 5), resulting in more faithful reconstructions.

In Figure 1(c), we show the isosurface of a trilinear interpolant, and in Table 1, we report quantitative results associated with trilinear interpolation. The “ground truth” trilinear interpolant could be considered as the upper bound of all methods that follow the trilinearity assumption. Therefore, our method outperforming the trilinear interpolant further proves that NMC is fundamentally superior at feature-preserving isosurface extraction.

Organic shapes. In Figure 10, we show that when the ground truth shape has smooth undulations, our method is still able to reconstruct the surface details better than MC33. For a more comprehensive test on organic shapes, we compare the various methods on 100 meshes of human body shapes from the FAUST dataset [Bogo et al. 2014]. The quantitative results in Table 2 show that NMC and its variant can learn to predict both smooth and sharp features well, outperforming both MC33 and Lopes2003. Augmenting the training set with more organic shapes should further improve performance on such inputs, since our method is data-driven.

Varying input grid resolutions. In Figure 11, we show how MC33 and NMC perform as the input SDF resolutions vary from 8³ to 128³, where we recall that our network was trained on meshes obtained at 32³ and 64³ resolutions. It is clear that our method can easily adapt to higher-resolution inputs, but degrades in reconstruction quality at the lower end. This is expected since as the cube size grows relative to the shape, the geometric varieties inside the cubes would surpass the set of topological cases covered by NMC. Nevertheless, NMC appears to consistently outperform MC33 at all resolutions, up to at least 128³. As the resolution continues to grow however, the difference between NMC and MC33 will diminish since the topological cases per cube would be much simplified.

Mesh extraction from binary voxels. When the inputs are binary voxels instead of signed distances, the isosurface extraction task becomes significantly more difficult, since voxel occupancies possess considerably less information. Not only are the point-to-surface

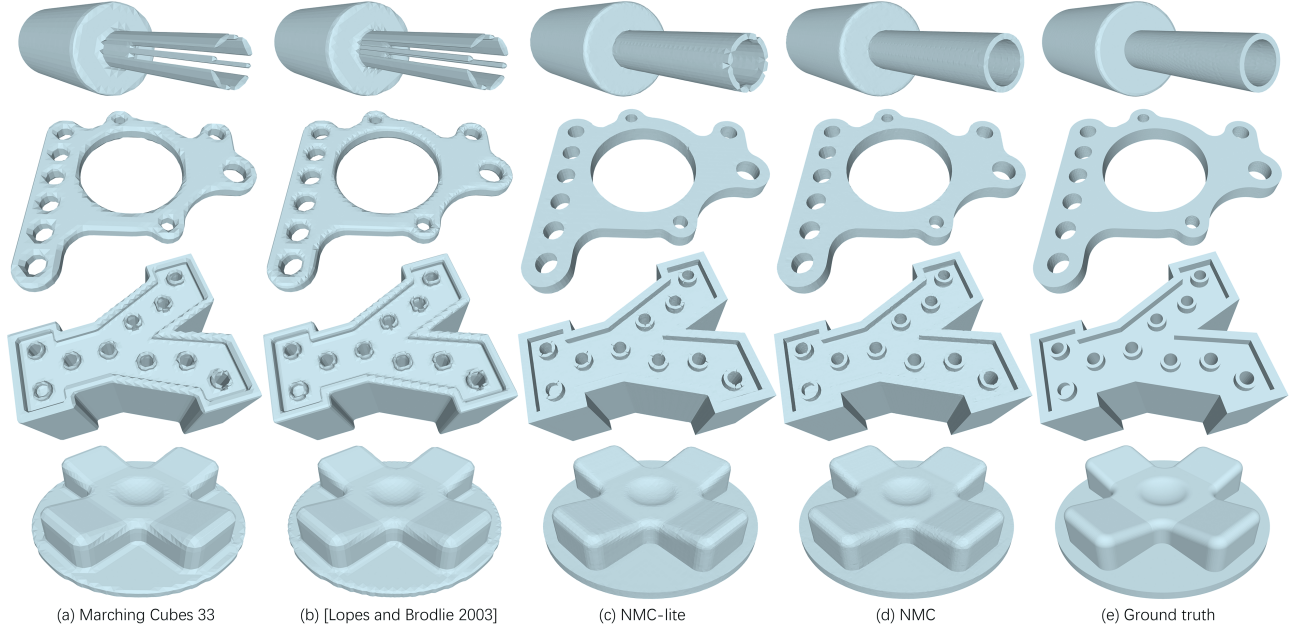


Fig. 9. Results of reconstructing 3D meshes from SDF grid inputs at 64^3 resolution. The shapes in the first two rows are from the ABC test set, and the last two rows from Thingi10K. More results and their mesh tessellations can be found in the supplementary material.

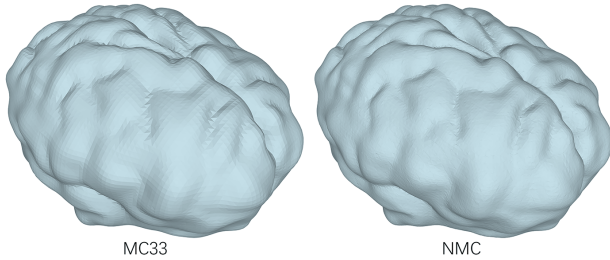


Fig. 10. Reconstruction results on a brain model (in Thingi10K) with smooth features by MC33 and NMC, from SDF inputs. NMC preserves the surface details (especially around the valley areas) better.

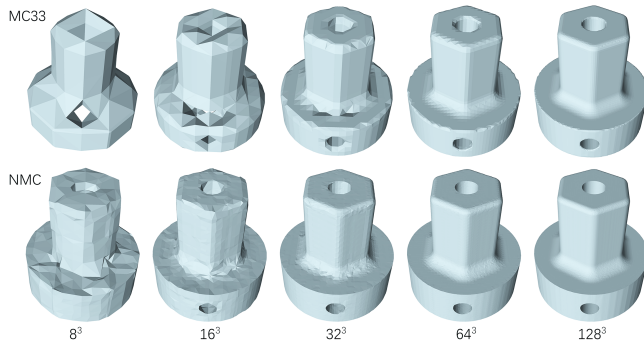


Fig. 11. Results of reconstructing 3D mesh shapes from SDF inputs as the input grid resolutions vary. The holes in the MC33 results are due to incorrectly predicted topological cases. NMC consistently outperforms MC33 at all input resolutions, up to 128^3 , but with a “diminishing margin”.

Table 3. Quantitative comparisons on ABC test set with binary voxel input.

64^3 resolution	CD($\times 10^5$)↓	F1↑	NC↑	ECD($\times 10^5$)↓	EF1↑	#V	#T
MC33	26.860	0.085	0.921	11.196	0.018	5,826.08	11,655.52
Lopes2003	26.829	0.084	0.921	14.601	0.017	23,302.73	46,608.90
Trilinear	26.826	0.084	0.921	14.866	0.017	-	-
NMC-lite	9.302	0.443	0.930	0.559	0.365	22,185.94	42,915.64
NMC	9.341	0.438	0.931	0.528	0.356	42,043.03	84,087.85
32^3 resolution	CD($\times 10^5$)↓	F1↑	NC↑	ECD($\times 10^5$)↓	EF1↑	#V	#T
MC33	9.636	0.036	0.882	11.764	0.018	1,532.70	3,065.30
Lopes2003	9.632	0.036	0.883	14.723	0.017	6,130.84	12,261.58
Trilinear	9.641	0.035	0.884	14.820	0.017	-	-
NMC-lite	5.909	0.237	0.871	0.901	0.112	5,236.79	9,975.67
NMC	6.029	0.232	0.871	0.910	0.109	9,469.84	18,933.65

distances lost in the occupancies, but the signs could also be inaccurate: a point outside the shape in the SDF grid may become “inside” in the voxel grid. One can observe a quality drop from the visual results shown in Figure 12. Even our method cannot always produce faithful reconstructions due to the ambiguities, e.g., the rod in the first row could be rectangular or circular, and the edges in the second row could be smooth or sharp - both would yield identical voxel grids. However, our learning-based approach is able to narrow down the possible geometries and topologies by observing local neighborhoods. As shown by the quantitative results in Table 3, our method outperforms other MC variants and the trilinear interpolant on all measures, except for NC, by a substantial margin.

Generalizability. To demonstrate generalizability of our networks, which were trained on ABC, we test them on the first 2,000 valid shapes from Thingi10K, using exactly the same network weights as those in the previous experiments. Tables 4 and 5 show quantitative comparison results on SDF and binary voxel inputs, respectively. Some qualitative results are given in Figures 9 and 12. We

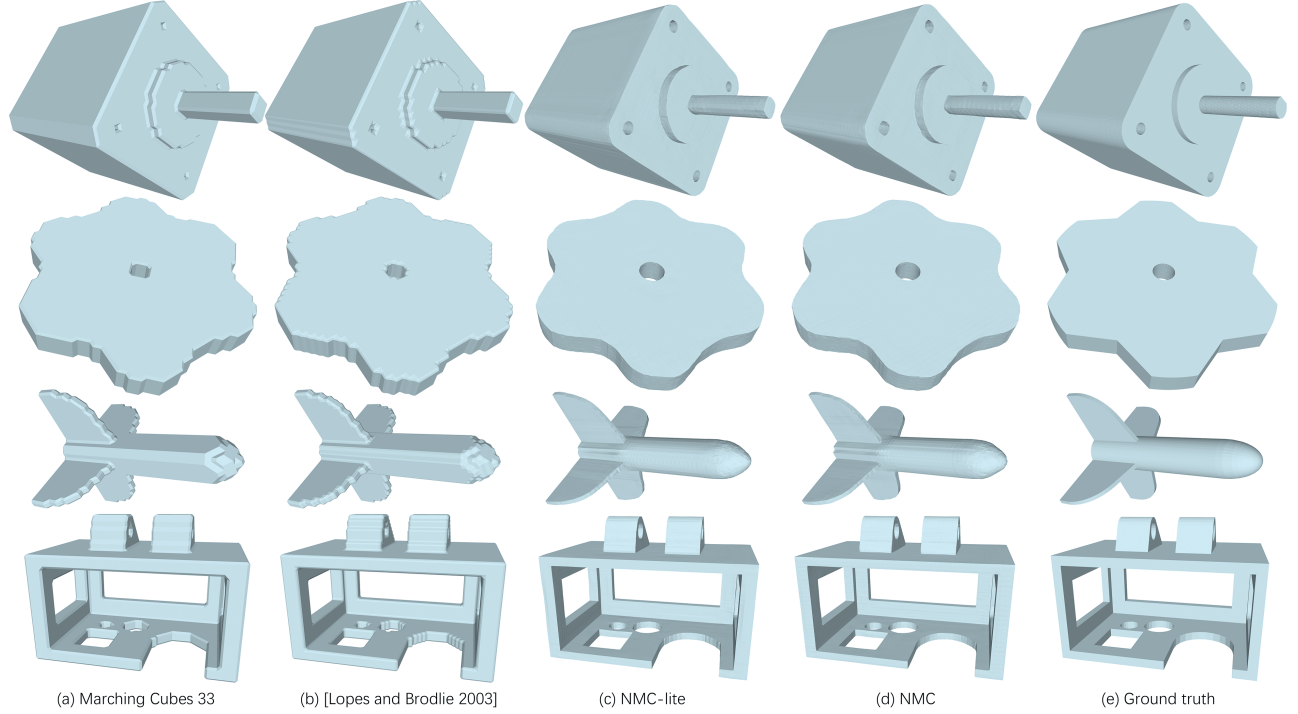


Fig. 12. Results of reconstructing 3D meshes from binary voxel/occupancy inputs at 64^3 resolution. The shapes in the first two rows are from the ABC test set, and the last two rows from Thingi10K. More results and their mesh tessellations can be found in the supplementary material.

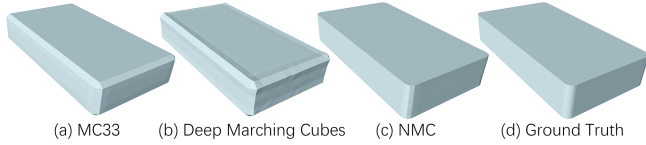


Fig. 13. Comparing NMC with MC33 and Deep Marching Cubes (DMC) [Liao et al. 2018] on feature preservation.

can observe a similar performance boost over the other methods in comparison. Note however that in Table 5, our method does not significantly outperform other methods at the 32^3 input voxel resolution. This may be due to NMC being overfit to the ABC training set, since the shape resolution (32^3) is getting close to the size of the receptive field of our voxel processing network (15^3).

Comparison to DMC. In Figure 13, we compare NMC with DMC [Liao et al. 2018] on feature-preserving mesh reconstruction. Since the network architecture of DMC is not designed to perform general isosurface extraction, we train their network to *overfit* on a single input shape with 65,536 uniformly sampled points as supervision. As we can see, even with such an overfitting, DMC is still unable to recover sharp features, which is mainly due to its adoption of only few classical MC tessellations representing simple topologies. Related to this, while DMC is trained to minimize point-to-triangle distances, it does not provide the tessellations to support sharp edges. As a result, the reconstructed geometry near edges is “bulging” in order to minimize distances to the training points.

Table 4. Quantitative comparison results on Thingi10K with SDF input.

64^3 resolution	CD($\times 10^5$)↓	F1↑	NC↑	ECD($\times 10^2$)↓	EF1↑	#V	#T
MC33	3.195	0.795	0.945	3.763	0.099	5,517.51	11,044.35
Lopes2003	3.084	0.805	0.953	4.567	0.087	22,224.23	44,135.98
Trilinear	3.076	0.811	0.956	5.211	0.084	-	-
NMC-lite	2.470	0.893	0.972	0.330	0.722	22,991.80	44,109.17
NMC	2.477	0.893	0.972	0.312	0.722	40,951.73	81,910.41
32^3 resolution	CD($\times 10^4$)↓	F1↑	NC↑	ECD($\times 10^2$)↓	EF1↑	#V	#T
MC33	10.519	0.540	0.882	4.046	0.040	1,284.98	2,569.73
Lopes2003	10.473	0.547	0.893	4.596	0.038	5,163.28	10,281.15
Trilinear	10.431	0.555	0.897	5.180	0.037	-	-
NMC-lite	8.425	0.807	0.935	0.600	0.542	5,423.92	10,263.13
NMC	8.454	0.808	0.933	0.596	0.539	9,161.94	18,327.88

Table 5. Quantitative comparisons on Thingi10K with binary voxel input.

64^3 resolution	CD($\times 10^5$)↓	F1↑	NC↑	ECD($\times 10^2$)↓	EF1↑	#V	#T
MC33	25.538	0.069	0.907	7.411	0.017	5,939.62	11,881.67
Lopes2003	25.526	0.068	0.908	11.948	0.015	23,757.44	47,517.48
Trilinear	25.510	0.068	0.909	12.598	0.015	-	-
NMC-lite	6.055	0.495	0.923	0.606	0.328	22,540.88	43,272.05
NMC	6.108	0.493	0.923	0.602	0.314	40,430.06	80,861.75
32^3 resolution	CD($\times 10^4$)↓	F1↑	NC↑	ECD($\times 10^2$)↓	EF1↑	#V	#T
MC33	9.247	0.028	0.865	8.632	0.017	1,553.93	3,107.50
Lopes2003	9.246	0.028	0.867	12.344	0.015	6,215.99	12,431.69
Trilinear	9.256	0.028	0.867	12.709	0.015	-	-
NMC-lite	9.998	0.258	0.852	0.946	0.096	5,261.82	9,971.62
NMC	10.177	0.256	0.852	0.957	0.093	9,043.78	18,083.90

Input and output complexities. When making comparisons, the input resolutions to all methods are identical, but the output complexities do vary, as shown in Tables 1-5, in terms of the average triangle and vertex counts. With the same tessellation templates, hence

comparable output complexities, NMC-lite outperforms Lopes2003 on all fronts, both quantitatively and qualitatively, offering clear evidence for the effectiveness of our data-driven approach for isosurfacing. The new tessellation templates designed for NMC are more refined, resulting in higher triangle counts, but also improved reconstruction quality, as shown in Figures 9 and 12.

NMC vs NMC-lite. Quantitatively, the performances of NMC and NMC-lite are quite similar, proving the robustness of our representation design. However, the visual quality of NMC results tends to be better than that of NMC-lite, at the expense of almost doubling the triangle counts. Thus, if a lower output complexity is desired, one may choose NMC-lite over NMC. But since NMC-lite employs simpler tessellation templates, it may not be able to recover specific fine shape features, such as the thin structures in the examples from the first and third rows of Figure 9, where the cubes with Case 3.2 were not well reconstructed. Also, we have observed that the *triangle quality* resulting from NMC is generally better than that from NMC-lite (e.g., see Figure 1), since the NMC tessellation templates were designed to better avoid thin/sliver triangles.

Training and testing times. Network training takes about 3 days on one Nvidia RTX 2080 Ti GPU for SDF processing and 2 days for binary voxel inputs. We tested inference time on the entire ABC test set with 64^3 inputs: the average is 0.006 second per shape for MC33 (implemented in scikit-image [Van der Walt et al. 2014]), and 0.762 second for NMC (with 0.719s spent on network forwarding in PyTorch [Paszke et al. 2019] and 0.042s for meshing in Cython [Behnel et al. 2011]). Note that currently, our network is still quite crude, as we prioritize accuracy over speed. Possible speed-up could be achieved via neural architecture search and network pruning.

Noisy inputs. Finally, we show that NMC can also learn to extract *clean* meshes from noisy grid inputs when the network is trained on such data, such as those generated by current neural implicit models [Chen and Zhang 2019; Mescheder et al. 2019; Park et al. 2019]. To test this capability, we run a state-of-the-art neural implicit model, SIREN [Sitzmann et al. 2020], on the ABC dataset to fit each shape, but with only 4,096 training points per shape. The sparsity of the training points makes the output implicit fields necessarily noisy, as can be observed from Figures 14(a-b).

In our previous experiments, we trained NMC on clean data from ABC and assumed that the testing inputs were also clean. A model trained this way may fail when the input is noisy, as shown in Figure 14(c). To remedy this, we divide the noisy inputs into 80% training and 20% testing as before, and use the noisy inputs to train the NMC model from scratch. The re-trained NMC improves significantly on inputs from the noisy test set, as shown in Figure 14, demonstrating that our method can adapt to different inputs (such as voxels and noisy data), if trained on them.

Table 6 shows a quantitative comparison involving NMC trained on clean vs. noisy data. We note that Chamfer Distance (CD) is rather sensitive to outliers, e.g., SIREN may generate blobs in the empty region that are far away from the shape, which can impact CD heavily. In comparison, F1 is a more robust quality measure to outliers, as discussed in [Tatarchenko et al. 2019].

Table 6. Quantitative comparison on ABC test set with noisy SDF input.

64^3 resolution	CD($\times 10^5$)↓	F1↑	NC↑	ECD($\times 10^2$)↓	EF1↑
MC33	16.611	0.710	0.942	3.360	0.100
Lopes2003	16.545	0.714	0.947	3.692	0.093
NMC (trained on clean data)	15.340	0.769	0.941	0.574	0.502
NMC (trained on noisy data)	15.627	0.802	0.951	0.359	0.640

5 CONCLUSION, LIMITATION, AND FUTURE WORK

In this paper, we show for the first time that the mesh reconstruction quality by Marching Cubes (MC), one of the most classical algorithms in computer graphics, can be significantly boosted by machine learning. In Neural Marching Cubes (NMC), we introduce the first MC-based approach capable of recovering sharp geometric features without requiring additional inputs, such as normal information. Trained on automatically generated “ground-truth” meshes, our method shows superior performance in preserving various geometric features such as sharp/soft edges, corners, thin structures, etc., compared to other isosurfacing algorithms that take uniform grids of signed distances or binary occupancies as inputs. We also designed an efficient parameterization to represent a triangle mesh, compatible with neural processing, so that our NMC network can directly output the meshes without postprocessing.

The main limitation of our method in terms of isosurfacing is its sensitivity to rotation, as shown in Figure 15. This is mainly due to the dataset we train the network on, as the shapes in the ABC dataset are mostly aligned with the coordinate axes. This data characteristic also motivated our definition of the smoothness term in Eq. (7). Performing random rotation augmentation on the training data is a viable solution, but would require longer training time and larger networks to fit. Second, as we follow the common assumption in MC that if the two vertices of a cube edge have different signs, then there is one and only one intersection point, several topological cases (as shown in Figure 16) cannot be represented. Adding more intersection points should cover most of such cases, and the numbers and the positions of the edge vertices can be learned from data.

Another limitation is that we do not have a built-in mechanism to avoid self-intersections in the output meshes. When testing on 64^3 SDF inputs from ABC, 32.7% of the meshes produced by NMC contain self-intersections, but they involve only 0.0086% of the triangles, which translate to about 7.39 triangles or around two separate intersections, per shape. For NMC-lite, the corresponding numbers are 29.6%, 0.0078%, and 3.40, respectively. Most of the intersections happen in cases where the structure in a cube cannot be represented by our tessellation templates, such as those in Figure 16(b). Last but not the least, our current model does not allow the learning of an arbitrary *tessellation style*, e.g., meshes whose triangles are all close to being equilaterals. The challenge is on how to prepare the proper training meshes to work under our designed templates.

Our proposed representation is not constrained to isosurfacing. It is a general shape representation that can be adopted to other tasks such as shape upsampling and generation, pointing to a worthy direction for future work. On the other hand, even when the input is a uniform grid, the output mesh does not have to be uniform. A simple plane only requires a few triangles to model, but a curved

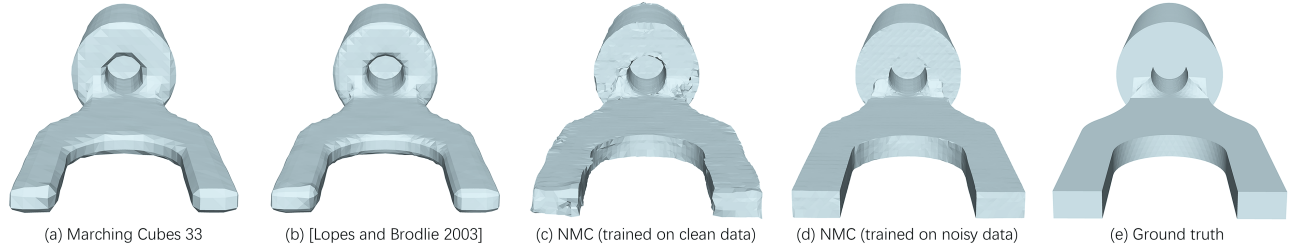


Fig. 14. Results of reconstructing 3D meshes from a noisy SDF grid input at 64^3 resolution.

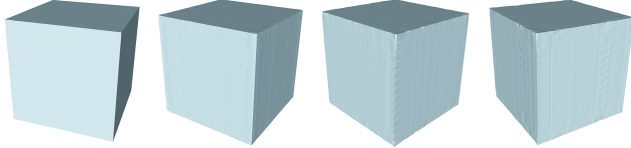


Fig. 15. NMC may produce artifacts when the input is oriented at an “unusual” angle relative to the training shapes. From left to right: reconstructions of a cube that is axis-aligned, then rotated by $\frac{1}{14}\pi$, $\frac{2}{14}\pi$, and $\frac{3}{14}\pi$.

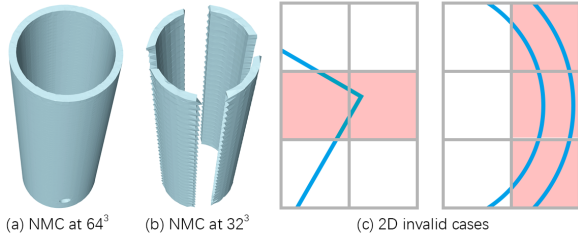


Fig. 16. NMC cannot account for certain topological cases (deemed “invalid”), e.g., multiple intersections along an edge as highlighted in red (c). The reconstruction failure in (b) is due to similar invalid cases in 3D.

surface needs more. Therefore, learning to adaptively allocate vertices and triangles according to feature complexity could yield more efficient algorithms and control the explosion of triangle counts in MC, as reported in BSP-NET [Chen et al. 2020].

ACKNOWLEDGMENTS

We thank the reviewers for their valuable comments, and Thomas Funkhouser for helpful discussions. This work was supported in part by NSERC (no. 611370) and gift funds from Google and Adobe.

REFERENCES

- Panos Achlioptas, Olga Diamanti, Ioannis Mitliagkas, and Leonidas J. Guibas. 2018. Learning representations and generative models for 3D point clouds. In *ICML*.
 Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. 2011. Cython: The best of both worlds. *Computing in Science & Engineering* (2011).
 Federica Bogo, Javier Romero, Matthew Loper, and Michael J. Black. 2014. FAUST: Dataset and evaluation for 3D mesh registration. In *CVPR*.
 Zhiqin Chen, Andrea Tagliasacchi, and Hao Zhang. 2020. BSP-Net: generating compact meshes via binary space partitioning. In *CVPR*.
 Zhiqin Chen and Hao Zhang. 2019. Learning implicit fields for generative shape modeling. In *CVPR*.
 Evgeni Chernyaev. 1995. *Marching cubes 33: construction of topologically correct isosurfaces*. Technical Report CN/95-17. CERN.
 Christopher B. Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 2016. 3D-R2N2: a unified approach for single and multi-view 3D object reconstruction. In *ECCV*.

- Paolo Cignoni, Fabio Ganovelli, Claudio Montani, and Roberto Scopigno. 2000. Reconstruction of topologically correct and adaptive trilinear isosurfaces. *Computers & Graphics* (2000).
 Lis Custodio, Tiago Etienne, Sinesio Pesco, and Claudio Silva. 2013. Practical considerations on marching cubes 33 topological correctness. *Computers & Graphics* (2013).
 Angela Dai and Matthias Nießner. 2019. Scan2Mesh: from unstructured range scans to 3D meshes. In *CVPR*.
 Boyang Deng, Kyle Genova, Soroosh Yazdani, Sofien Bouaziz, Geoffrey Hinton, and Andrea Tagliasacchi. 2020. Cvxnet: learnable convex decomposition. In *CVPR*.
 Tamal K. Dey and Joshua A. Levine. 2008. Delaunay meshing of isosurfaces. *The Visual Computer* (2008).
 Akio Doi and Akio Koide. 1991. An efficient method of triangulating equi-valued surfaces by using tetrahedral cells. *IEICE Transactions on Information and Systems* (1991).
 Martin J. Dürst. 1988. Letters: additional reference to marching cubes. In *SIGGRAPH*.
 Tiago Etienne, Luis Gustavo Nonato, Carlos Scheidegger, Julien Tienry, Thomas J. Peters, Valerio Pascucci, Robert M. Kirby, and Cláudio T. Silva. 2011. Topology verification for isosurface extraction. *TVCG* (2011).
 Haoqiang Fan, Hao Su, and Leonidas J. Guibas. 2017. A point set generation network for 3D object reconstruction from a single image. In *CVPR*.
 Jun Gao, Wenzheng Chen, Tommy Xiang, Clement Fuji Tsang, Alec Jacobson, Morgan McGuire, and Sanja Fidler. 2020. Learning deformable tetrahedral meshes for 3D reconstruction. In *NeurIPS*.
 Lin Gao, Jie Yang, Tong Wu, Hongbo Fu, Yu-Kun Lai, and Hao Zhang. 2019. SDM-NET: Deep Generative Network for Structured Deformable Mesh. *ACM Transactions on Graphics* (2019).
 Thibault Groueix, Matthew Fisher, Vladimir G. Kim, Bryan Russell, and Mathieu Aubry. 2018. AtlasNet: a papier-mâché approach to learning 3D surface generation. In *CVPR*.
 Christian Häne, Shubham Tulsiani, and Jitendra Malik. 2017. Hierarchical surface prediction for 3D object reconstruction. In *3DV*.
 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *CVPR*.
 Tao Ju, Frank Losasso, Scott Schaefer, and Joe Warren. 2002. Dual contouring of Hermite data. In *Computer Graphics and Interactive Techniques*.
 Leif P. Kobbelt, Mario Botsch, Ulrich Schwanke, and Hans-Peter Seidel. 2001. Feature Sensitive Surface Extraction from Volume Data. In *SIGGRAPH*.
 Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. 2019. ABC: a big CAD model dataset for geometric deep learning. In *CVPR*.
 Jiabao Lei and Kui Jia. 2020. Analytic marching: an analytic meshing solution from deep implicit surface networks. In *ICML*.
 Thomas Lewiner, Hélio Lopes, Antônio Wilson Vieira, and Geovan Tavares. 2003. Efficient implementation of marching cubes' cases with topological guarantees. *Journal of Graphics Tools* (2003).
 Yiyi Liao, Simon Donne, and Andreas Geiger. 2018. Deep marching cubes: learning explicit surface representations. In *CVPR*.
 Cheng Lin, Changjian Li, Yuan Liu, Nenglu Chen, Yi-King Choi, and Wenping Wang. 2021. Point2Skeleton: Learning Skeletal Representations from Point Clouds. In *CVPR*.
 Hsueh-Ti Derek Liu, Vladimir G. Kim, Siddhartha Chaudhuri, Noam Aigerman, and Alec Jacobson. 2020. Neural subdivision. *ACM Transactions on Graphics* (2020).
 Adriano Lopes and Ken Brodlie. 2003. Improving the robustness and accuracy of the marching cubes algorithm for isosurfacing. *TVCG* (2003).
 William E. Lorensen and Harvey E. Cline. 1987. Marching cubes: a high resolution 3D surface construction algorithm. In *SIGGRAPH*.
 Sergey V. Matveyev. 1994. Approximation of isosurface in the marching cube: ambiguity problem. In *IEEE Visualization*.
 Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. 2019. Occupancy networks: learning 3D reconstruction in function space.

Table 7. Comparison of different smoothness terms on the ABC test set. The underlined superscripts show rankings of the quantitative performances, where the overall best performing row is highlighted in bold.

64 ³ resolution	CD($\times 10^5$) $\downarrow$	F1 $\uparrow$	NC $\uparrow$	ECD($\times 10^2$) $\downarrow$	EF1 $\uparrow$
Eq. (8)	<u>4</u> 9.355	<u>1</u> 0.445	<u>3</u> 0.932	<u>5</u> 0.633	<u>4</u> 0.328
Eq. (9), $\gamma = 100$	<u>2</u> 9.329	<u>3</u> 0.435	<u>2</u> 0.932	<u>4</u> 0.615	<u>3</u> 0.353
Eq. (10), $\gamma = 100$	<u>6</u> 9.539	<u>3</u> 0.435	<u>2</u> 0.933	<u>3</u> 0.612	<u>5</u> 0.320
Eq. (11), $\gamma = 100$	<u>5</u> 9.518	<u>5</u> 0.434	<u>1</u> 0.934	<u>2</u> 0.562	<u>6</u> 0.313
Eq. (12), $\gamma=10$	<u>3</u> 9.341	<u>2</u> 0.438	<u>5</u> 0.931	<u>1</u> 0.528	<u>2</u> 0.356
Eq. (12), $\gamma = 100$	<u>1</u> 9.327	<u>6</u> 0.427	<u>6</u> 0.923	<u>6</u> 0.669	<u>1</u> 0.359

In CVPR.

Gregory M. Nielson. 2003. On marching cubes. *TVCG* (2003).

Gregory M. Nielson and Bernd Hamann. 1991. The asymptotic decoder: resolving the ambiguity in Marching Cubes. In *IEEE Visualization*.

Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. 2019. DeepSDF: Learning continuous signed distance functions for shape representation. In *CVPR*.

Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NeurIPS*.

Edoardo Remelli, Artem Lukoianov, Stephan R. Richter, Benoît Guillard, Timur Bagautdinov, Pierre Baque, and Pascal Fua. 2020. MeshSDF: differentiable iso-surface extraction. *NeurIPS* (2020).

Scott Schaefer, Tao Ju, and Joe Warren. 2007. Manifold dual contouring. *TVCG* (2007).

Scott Schaefer and Joe Warren. 2004. Dual marching cubes: primal contouring of dual grids. In *Pacific Conference on Computer Graphics and Applications (PG)*.

John Schreiner, Carlos E. Scheidegger, and Claudio T. Silva. 2006. High-quality extraction of isosurfaces from regular and irregular grids. *TVCG* (2006).

Vincent Sitzmann, Julien N.P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. 2020. Implicit Neural Representations with Periodic Activation Functions. In *NeurIPS*.

Maxim Tatarchenko, Stephan R Richter, René Ranftl, Zhuwen Li, Vladlen Koltun, and Thomas Brox. 2019. What do single-view 3d reconstruction networks learn?. In *CVPR*.

Stefan Van der Walt, Johannes L Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D Warner, Neil Yager, Emmanuelle Goullart, and Tony Yu. 2014. scikit-image: image processing in Python. *PeerJ* (2014).

Allen Van Gelder and Jane Wilhelms. 1994. Topological considerations in isosurface generation. *ACM Transactions on Graphics* (1994).

Gokul Varadhan, Shankar Krishnan, Young J. Kim, and Dinesh Manocha. 2003. Feature-sensitive subdivision and isosurface reconstruction. In *IEEE Visualization*.

Nanyang Wang, Yinda Zhang, Zhuwen Li, Yanwei Fu, Wei Liu, and Yu-Gang Jiang. 2018. Pixel2Mesh: generating 3D mesh models from single RGB images. In *ECCV*.

Peng-Shuai Wang, Yang Liu, and Xin Tong. 2016. Mesh Denoising via Cascaded Normal Regression. *ACM Transactions on Graphics* (2016).

Weiye Wang, Duygu Ceylan, Radomir Mech, and Ulrich Neumann. 2019. 3DN: 3D deformation network. In *CVPR*.

Jiajun Wu, Chengkai Zhang, Tianfan Xue, Bill Freeman, and Josh Tenenbaum. 2016. Learning a probabilistic latent space of object shapes via 3D generative-adversarial modeling. In *NeurIPS*.

Geoff Wyvill, Craig McPheeters, and Brian Wyvill. 1986. Data structure for soft objects. *The Visual Computer* (1986).

Kangxue Yin, Zhiqin Chen, Hui Huang, Daniel Cohen-Or, and Hao Zhang. 2019. LOGAN: Unpaired Shape Transform in Latent Overcomplete Space. *ACM Transactions on Graphics* (2019).

Lequan Yu, Xianzhi Li, Chi-Wing Fu, Daniel Cohen-Or, and Pheng-Ann Heng. 2018. PU-Net: Point Cloud Upsampling Network. In *CVPR*.

Nan Zhang, Wei Hong, and Arie Kaufman. 2004. Dual contouring with topology-preserving simplification using enhanced cell representation. In *IEEE Visualization*.

Qingnan Zhou and Alec Jacobson. 2016. Thing10k: a dataset of 10,000 3d-printing models. *arXiv preprint arXiv:1605.04797* (2016).

A COMPARISON OF DIFFERENT SMOOTHNESS TERMS

Due to considerable ambiguities in possible shapes represented by binary voxels, we need an extra smoothness term to regularize the generated surfaces. We reuse notations from Section 3.5 for E_s , F_s^u , and H_s^u , and $\mathbb{1}(\cdot)$. Further, let $F_s^{uv} = F_s^v - F_s^u$, $H_s^{uv} = H_s^v - H_s^u$, $[F_s^u]_x$

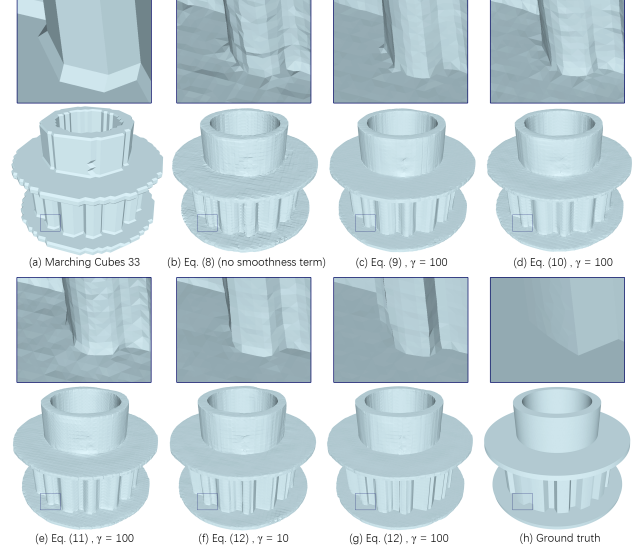


Fig. 17. Visual comparisons of different smoothness terms on ABC test set with binary voxel input at 64³ resolution.

be the x coordinate of F_s^u , and $[F_s^{uv}]_y = \sqrt{[F_s^{uv}]_y^2 + [F_s^{uv}]_z^2}$. We have experimented with the following settings:

$$L_{smooth}^{(1)} = 0 \quad (\text{No smoothness term}). \quad (8)$$

$$L_{smooth}^{(2)} = \mathbb{E}_{s \sim \mathcal{S}} \sum_{(u,v) \in E_s} \|F_s^{uv} - H_s^{uv}\|_2^2. \quad (9)$$

$$L_{smooth}^{(3)} = \mathbb{E}_{s \sim \mathcal{S}} \sum_{(u,v) \in E_s} (L_y^x + L_z^x + L_x^y + L_z^y + L_x^z + L_y^z), \quad (10)$$

$$\text{where } L_y^x = ([H_s^{uv}]_x \cdot |[F_s^{uv}]_y| - [F_s^{uv}]_x \cdot |[H_s^{uv}]_y|)^2.$$

$$L_{smooth}^{(4)} = \mathbb{E}_{s \sim \mathcal{S}} \sum_{(u,v) \in E_s} (L_{yz}^x + L_{xz}^y + L_{xy}^z), \quad (11)$$

$$\text{where } L_{yz}^x = ([H_s^{uv}]_x \cdot [F_s^{uv}]_{yz} - [F_s^{uv}]_x \cdot [H_s^{uv}]_{yz})^2.$$

$$L_{smooth}^{(5)} = \mathbb{E}_{s \sim \mathcal{S}} \sum_{(u,v) \in E_s} \|\mathbb{1}(|F_s^{uv}| < \sigma) \cdot H_s^{uv}\|_2^2. \quad (12)$$

The smoothness term $L_{smooth}^{(2)}$ in Eq. (9) minimizes the differences between the edge gradients on the predicted mesh and those on the GT. $L_{smooth}^{(3)}$ and $L_{smooth}^{(4)}$ try to preserve the absolute angles of the edges. In an ideal situation, $[H^{uv}]_x/[F^{uv}]_y = [F^{uv}]_x/[F^{uv}]_y$, therefore $[H^{uv}]_x \cdot |[F^{uv}]_y| - [F^{uv}]_x \cdot |[H^{uv}]_y| = 0$. Eq. (10) minimizes the squared error of such terms. Eq. (11) is a variant of Eq. (10), while Eq. (12) is equivalent to Eq. (7) in Section 3.5.

The performances of the different smoothness terms are shown in Table 7 and exhibited visually in Figure 17. Based on the visual results and overall quantitative performances, we have adopted $L_{smooth}^{(5)}$ in our work, with $\gamma = 10$ and $\sigma = 0.0002$.